Unit IV

4. Database Handling Using PHP with MySQL

4.1 Introduction to MySQL: Database terms, Data Types
4.2 Accessing MySQL –Using MySQL Client and Using phpMyAdmin
4.3 MySQL Commands
4.4 Using PHP with MySQL: PHP MySQL Functions
4.5 Connecting to MySQL and Selecting the Database
4.6 Executing Simple Queries
4.7 Retrieving Query Results
4.8 Counting Returned Records
4.9 Updating Records with PHP

4.1 Introduction to MySQL: Database terms, Data Types:

Introduction to MySQL:

MySQL is a popular **open-source relational database management system (RDBMS)** used to store, manage, and retrieve data efficiently. It works with PHP to build dynamic websites and applications.

1. Common Database Terms:

Term	Description
Database	A collection of organized data.
Table	Stores data in rows and columns.
Row	A single record in a table.
Column	A field that holds specific data (e.g., name, age).
Primary Key	Uniquely identifies each row.
Foreign Key	Links one table to another.
Query	A command used to interact with the database.

2. Common MySQL Data Types:

Data Type	Description
INT	Integer numbers(e.g., age, ID)
VARCHAR	Variable-length string(e.g., name, email)
TEXT	Large text data
DATE	Stores date(YYYY-MM-DD)
DATETIME	Stores date and time
FLOAT/DOUBLE	Decimal numbers
BOOLEAN/TINYINT(1)	True/False values(1 or 0)

Example:

sql

```
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(100),
```

```
        dob DATE,
        marks FLOAT
);
```

Summary:

MySQL stores data in structured form using tables.
Different data types are used based on the kind of data (numbers, text, date).
Understanding these basics is essential for building a strong database foundation.

4.2 Accessing MySQL –Using MySQL Client and Using phpMyAdmin:

Accessing MySQL:

You can access and manage MySQL databases in two main ways:

1. Using MySQL Client (Command Line Interface):

A **text-based interface** to interact with MySQL directly.
Useful for quick and powerful database management.

Steps:

1. Open Command Prompt or Terminal
2. Login to MySQL:
   bash
   mysql -u root –p
3. Enter your password
4. Now you can run SQL commands like:
   sql
   SHOW DATABASES;
   USE mydatabase;
   SELECT * FROM students;

Fast and flexible but requires knowledge of SQL.

2. Using phpMyAdmin (Web Interface):

A **browser-based GUI tool** for MySQL.
Easier for beginners and provides visual tools.

Steps:

1. Start your local server (XAMPP, WAMP, etc.)
2. Open browser and go to:
   http://localhost/phpmyadmin
3. Login with username/password (default: root / no password)
4. You can:
   Create/edit databases and tables
   Run SQL queries
   Import/export data

User-friendly and no SQL knowledge required for basic operations.

Summary Table:

Method	Interface	Best For
MySQL Client	Command-line	Developers, quick access
phpMyAdmin	Web-based GUI	Beginners, visual tools

4.3 MySQL Commands:

MySQL Commands:

MySQL commands are **SQL (Structured Query Language) statements** used to interact with a database to create, manage, and retrieve data from it.

1. Database Commands:

Create Database:

sql

```
CREATE DATABASE college;
```

Use Database:

sql

```
USE college;
```

Show Databases:

sql

```
SHOW DATABASES;
```

2. Table Commands:

Create Table:

sql

```
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    marks FLOAT
);
```

Show Tables:

sql

```
SHOW TABLES;
```

3. Data Manipulation Commands:

Insert Data:

sql

```
INSERT INTO students VALUES (1, 'Ayaan', 85.5);
```

Select Data:

sql

```
SELECT * FROM students;
```

Update Data:

sql

```
UPDATE students SET marks = 90 WHERE id = 1;
```

Delete Data:

sql

```
DELETE FROM students WHERE id = 1;
```

Summary Table:

Command	Purpose
`CREATE`	To create database/table
`INSERT`	To add new data
`SELECT`	To retrieve data
`UPDATE`	To modify existing data
`DELETE`	To remove data

4.4 Using PHP with MySQL: PHP MySQL Functions:

Using PHP with MySQL:

PHP can connect to MySQL to **store**, **retrieve**, and **manage data** from a database. This is useful for creating dynamic websites like login forms, contact forms, etc.

1. Connecting to MySQL:

php

```
$conn = mysqli_connect("localhost", "root", "", "college");
```

`localhost`: Server name
`root`: Username
`""`: Password (empty by default in local setup)
`"college"`: Database name

2. PHP MySQL Functions:

Function	Purpose
`musqli_connect()`	Connect to MySQL database
`mysqli_query()`	Execute SQL queries (SELECT, INSERT, etc.)
`mysqli_fetch_assoc()`	Fetch query result as an associative array
`mysqli_close()`	Close the database connection
`mysqli_error()`	Display error if query fails

3. Example: Insert & Display Data:

php

```
$conn = mysqli_connect("localhost", "root", "", "college");
$sql = "INSERT INTO students (id, name, marks) VALUES (1, 'Ayaan', 85)";
```

```
mysqli_query($conn, $sql);

$result = mysqli_query($conn, "SELECT * FROM students");
while($row = mysqli_fetch_assoc($result)) {
    echo $row['name'] . " - " . $row['marks'] . "<br>";
}

mysqli_close($conn);
```

Summary:

PHP uses `mysqli_` functions to:

Connect to a database
Run queries
Fetch results
Close the connection

4.5 Connecting to MySQL and Selecting the Database:

Connecting to MySQL and Selecting the Database in PHP:

To work with MySQL in PHP, you first need to **connect to the MySQL server** and then **select a database** to run queries on.

1. Using `mysqli_connect()` Function:

php

```
$conn = mysqli_connect("localhost", "root", "", "college");
```

"localhost" – MySQL server (usually local)
"root" – MySQL username
"" – Password (empty for local servers like XAMPP/WAMP)
"college" – Name of the database

2. Checking Connection:

php

```
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
echo "Connected successfully!";
```

This ensures the script stops if the connection fails and shows an error message.

3. Selecting a Database (Optional if included in connection):

You can also select the database separately:

php

```
mysqli_select_db($conn, "college");
```

Summary Table:

Function	Purpose
`mysqli_connect()`	Connects to MySQL server
`mysqli_select_db()`	Selects a database (if not passed in connect)
`mysqli_connect_error()`	Shows connection error if any

Example Output:

Connected successfully!

4.6 Executing Simple Queries:

Executing Simple Queries in PHP with MySQL:

To interact with a MySQL database from PHP, you use **SQL queries**. These queries can **insert, select, update, or delete** data from the database using PHP functions.

1. Using `mysqli_query()`:

This function is used to execute SQL queries in PHP:

php

```
mysqli_query($connection, $query);
```

2. Example: INSERT Query:

php

```
$conn = mysqli_connect("localhost", "root", "", "college");

$sql = "INSERT INTO students (id, name, marks) VALUES (1, 'Ayesha', 88)";
mysqli_query($conn, $sql);
```

This will insert a new record into the `students` table.

3. Example: SELECT Query:

php

```
$result = mysqli_query($conn, "SELECT * FROM students");

while($row = mysqli_fetch_assoc($result)) {
    echo $row['name'] . " - " . $row['marks'] . "<br>";
}
```

This retrieves all records and displays student name and marks.

4. Checking if Query was Successful:

php

```
if (mysqli_query($conn, $sql)) {
        echo "Query executed successfully";
} else {
        echo "Error: " . mysqli_error($conn);
}
```

Summary Table:

Function	Use
`mysqli_query()`	Executes any SQL command
`mysqli_fetch_assoc()`	Fetches result row as array (for SELECT)
`mysqli_error()`	Returns error message, if any

4.7 Retrieving Query Results:

Retrieving Query Results in PHP (MySQLi):

After running a `SELECT` query in PHP, you can retrieve and display data using **result-handling functions** like `mysqli_fetch_assoc()`.

1. Step-by-Step Process:

1. Run a SELECT Query:

php

```
$conn = mysqli_connect("localhost", "root", "", "college");

$sql = "SELECT * FROM students";
$result = mysqli_query($conn, $sql);
```

2. Check if Results Exist:

php

```
if (mysqli_num_rows($result) > 0) {
        // Data exists
}
```

3. Fetch and Display Results:

php

```
while($row = mysqli_fetch_assoc($result)) {
        echo "ID: " . $row['id'] . " - Name: " . $row['name'] . " - Marks: " .
    $row['marks'] . "<br>";
}
```

2. Common Result Functions:

Function	Description
`mysqli_query()`	Executes the SQL query
`mysqli_num_rows()`	Returns number of rows from SELECT result

`mysqli_fetch_assoc()`	Fetches a row as an associative array
`mysqli_fetch_array()`	Fetches row as both indexed and assoc. array

3. Example Output:

ID: 1 - Name: Ayesha - Marks: 88
ID: 2 - Name: Zaid - Marks: 92

Summary:

Use `SELECT` to retrieve data.
`mysqli_fetch_assoc()` is most commonly used for clean associative data.
Always check if data exists before fetching.

4.8 Counting Returned Records:

Counting Returned Records in PHP (MySQLi):

When you run a **`SELECT` query**, you can count how many records (rows) were returned using the **`mysqli_num_rows()`** function.

1. Step-by-Step Example:

php

```
$conn = mysqli_connect("localhost", "root", "", "college");

$sql = "SELECT * FROM students";
$result = mysqli_query($conn, $sql);

// Count records
$count = mysqli_num_rows($result);

echo "Total students: " . $count;
```

This displays the total number of rows fetched from the `students` table.

2. Function Used:

Function	Purpose
`mysqli_num_rows()`	Returns the number of rows in a result set

3. Use Case:

To show total results to the user
To check if any data was returned:

php

```
if (mysqli_num_rows($result) > 0) {
    echo "Data found!";
} else {
    echo "No data available.";
```

```
}
```

Summary:

`mysqli_num_rows()` is used **after a SELECT query**.
It helps in displaying totals or checking if any records were returned.

4.9 Updating Records with PHP:

Updating Records with PHP (MySQLi):

In PHP, you can **update existing records** in a MySQL database using the `UPDATE` SQL query and the `mysqli_query()` function.

1. Syntax of UPDATE Statement:

sql

```sql
UPDATE table_name SET column1 = value1 WHERE condition;
```

`SET` specifies the new value
`WHERE` filters which row(s) to update

2. PHP Example: Update a Record:

php

```php
$conn = mysqli_connect("localhost", "root", "", "college");

$sql = "UPDATE students SET marks = 95 WHERE id = 1";

if (mysqli_query($conn, $sql)) {
    echo "Record updated successfully";
} else {
    echo "Error updating record: " . mysqli_error($conn);
}
```

This updates the marks of the student with `id = 1`.

3. Points to Remember:

Always use a **WHERE clause** to avoid updating all rows.
Use **form input** for dynamic updates:

php

```php
$id = $_POST['id'];
$newMarks = $_POST['marks'];
$sql = "UPDATE students SET marks = $newMarks WHERE id = $id";
```

Summary Table:

Function	Use
`mysqli_query()`	Executes the update SQL query
`mysqli_error()`	Shows error if update fails